

INSEGUS

PAI-4. DevSecOps

Aseguramiento de la Cadena de Suministro en el Desarrollo de Aplicaciones Web

UNIVERSIDAD D SEVILLA
1505

Índice

1. **Contexto**
2. Problemas de seguridad
3. Objetivos y requisitos
4. Entrega

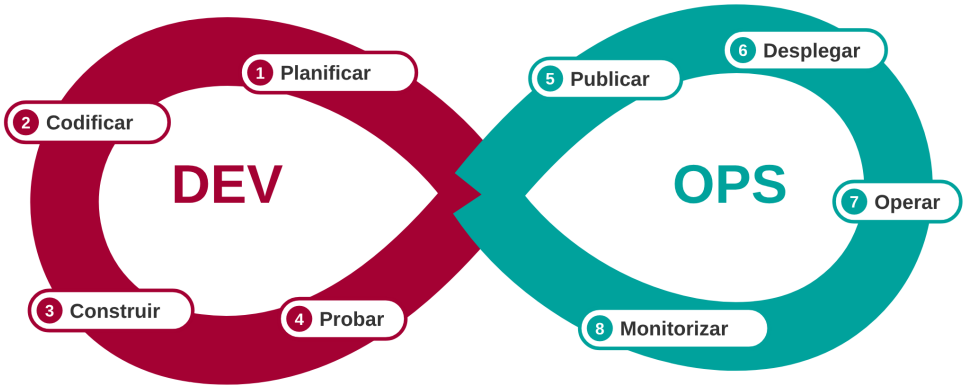
Contexto

DevOps: desarrollo y operaciones como un único ciclo

Antes: un equipo desarrollaba y otro distinto desplegaba y mantenía. Entregas grandes cada varios meses y muchos problemas al pasar a producción.

Con DevOps: el mismo equipo planifica, desarrolla, despliega y opera el software, y **automatiza** todo lo que puede.

Resultado: cambios pequeños y frecuentes, incluso varias veces al día, y fallos que se detectan y corrigen antes.



Contexto

CI/CD: la automatización que hace posible DevOps

CI
Integración continua

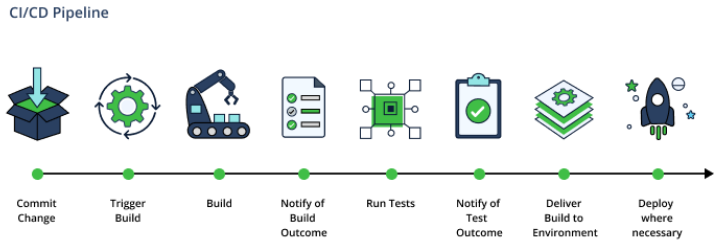
Cada cambio se integra en la rama principal y se **compila y prueba automáticamente**.

CD
Entrega continua

El software queda **siempre listo para desplegar**; el paso a producción lo aprueba una persona.

CD
Despliegue continuo

Si todas las pruebas pasan, **se despliega en producción** sin intervención humana.



El **pipeline actual del cliente**: cada commit dispara automáticamente el build, las pruebas, las notificaciones y el despliegue.

Ninguna de esas etapas comprueba la seguridad.

Contexto

Un pipeline se define como código, dentro del repositorio

1. Se describe en un **fichero del repositorio** (por ejemplo, .github/workflows/ci.yml).
2. Un **servidor de CI** (GitHub Actions, GitLab CI/CD, Jenkins) lo ejecuta en cada push o pull request.
3. Cada **job** es una etapa y cada **step**, una orden.
4. Si un paso falla, **el pipeline se detiene** y avisa al equipo.

```yaml
name: CI
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npm test
      - run: docker build -t app .
```

Ejemplo mínimo con GitHub Actions. En el PAI 4 añadiréis aquí los pasos de seguridad.

Contexto

El cliente: una empresa de desarrollo web con incidentes de seguridad

Situación

- Es cliente de **INSEGUS** y desarrolla aplicaciones web.
- Ha sufrido **varios incidentes**: vulnerabilidades que llegaron a producción sin detectarse.
- Ya trabaja con un **pipeline CI/CD** automatizado.
- Está migrando a **microservicios en contenedores**, gestionados con **Kubernetes** en la nube.

La pregunta del cliente

«¿Se deben los incidentes a nuestra forma de trabajar y a cómo comprobamos que se cumplen nuestras políticas de seguridad?»

Vuestra respuesta: integrar la seguridad dentro del pipeline (**DevSecOps**).

Contexto

En este PAI trabajáis como Yellow Team: quienes construyen el software



La **InfoSec Colour Wheel** reparte la seguridad por equipos: el **Red Team** ataca, el **Blue Team** defiende y el **Yellow Team** construye (desarrollo y arquitectura).

DevSecOps lleva la seguridad al Yellow Team: quienes construyen automatizan también las pruebas de seguridad. Es el terreno del **Green Team**, que combina construcción y defensa.

En el marco **NICE** del NIST (SP 800-181), estas tareas pertenecen a la categoría **Design and Development**, con roles como el de desarrollo de software seguro.

Índice

1. Contexto
2. **Problemas de seguridad**
3. Objetivos y requisitos
4. Entrega

Problemas de seguridad

Las vulnerabilidades no solo están en el código que escribimos

Código propio

Inyección SQL, XSS, credenciales escritas en el código…

Detección: SAST

Dependencias de terceros

Librerías con vulnerabilidades conocidas (CVE). La mayor parte del código de una aplicación moderna no lo escribe el equipo.

Detección: SCA

Configuración

Contenedores que se ejecutan como root, puertos abiertos, secretos en Dockerfiles o manifiestos de Kubernetes.

Detección: IaC

Aplicación en ejecución

Fallos que solo aparecen al ejecutarla: autenticación, sesiones, cabeceras de seguridad…

Detección: DAST

Problemas de seguridad

dependência transitiba de bibliotecas.

Ataques a la cadena de suministro: lo que no escribes tú también te afecta

2020 **SolarWinds**

Los atacantes comprometieron el sistema de build del software Orion e insertaron una puerta trasera en actualizaciones oficiales. Unos 18 000 clientes la instalaron.

2021 **Log4Shell**

Vulnerabilidad crítica en Log4j, una librería de registro de Java presente en millones de aplicaciones. Permitía ejecutar código en el servidor con una simple cadena de texto.

2024 **xz Utils**

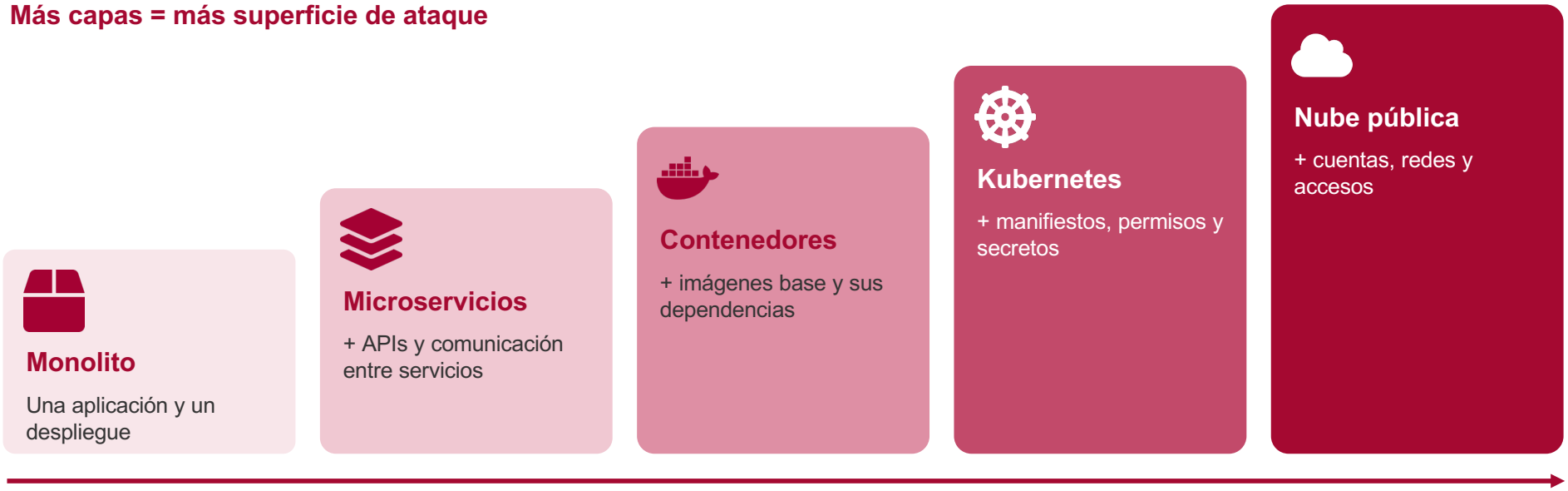
Un colaborador se ganó la confianza del proyecto durante unos dos años e introdujo una puerta trasera que afectaba a SSH. Se descubrió casi por casualidad.

Lo que trabajaréis en el PAI: **SCA** para controlar las dependencias y un **pipeline con pruebas automáticas y quality gates**.

Problemas de seguridad

La arquitectura del cliente multiplica la superficie de ataque

Más capas = más superficie de ataque



Problemas de seguridad

Shift left: cuanto antes se detecta una vulnerabilidad, más barato es corregirla



Coste de corregir una vulnerabilidad según dónde se detecta

En producción, corregir obliga a volver atrás, rehacer, probar y redesplegar… y puede que ya haya habido un incidente.

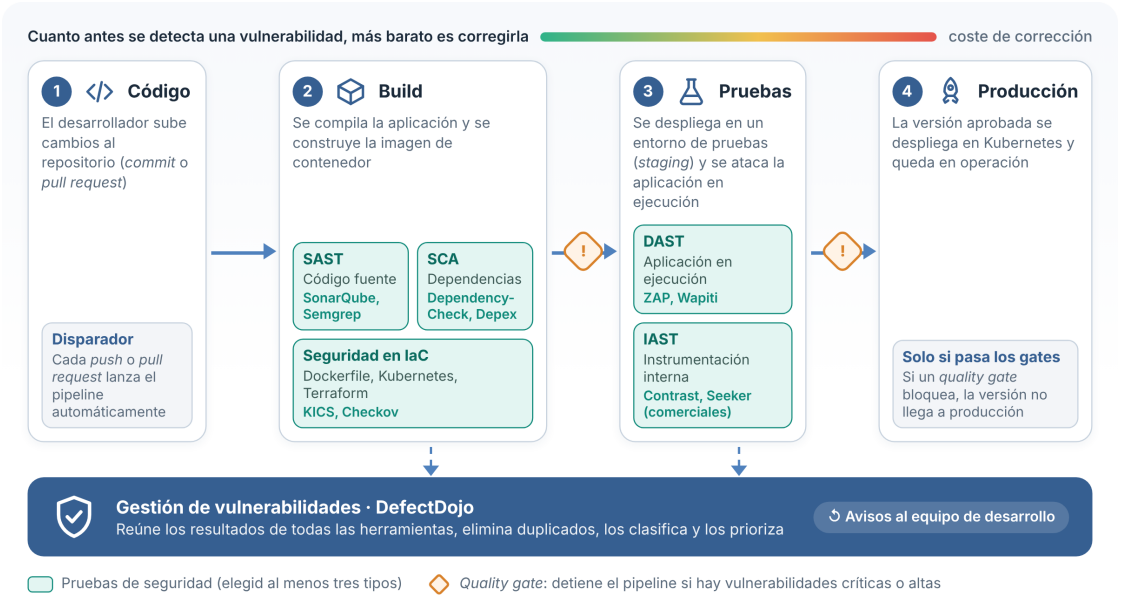
Con entregas diarias, una revisión manual al final no escala: **la seguridad tiene que ir dentro del pipeline, desde el primer commit**.

Índice

1. Contexto
2. Problemas de seguridad
3. **Objetivos y requisitos**
4. Entrega

Objetivos y requisitos

DevSecOps: pruebas de seguridad automáticas en cada etapa del pipeline



Objetivos y requisitos

Cinco tipos de pruebas de seguridad: elegid al menos tres

SCA

Software Composition Analysis

Dependencias de terceros con vulnerabilidades conocidas

Etapa: Build

Dependency-Check, Depex

SAST

Static Application Security Testing

El código fuente, sin ejecutarlo

Etapa: Build

SonarQube, Semgrep

DAST

Dynamic Application Security Testing

La aplicación en ejecución, atacándola desde fuera

Etapa: Pruebas

ZAP, Wapiti

IAST

Interactive Application Security Testing

La aplicación en ejecución, instrumentada desde dentro

Etapa: Pruebas

Contrast, Seeker (comerciales)

IaC

Infrastructure as Code

Dockerfile, Kubernetes, Terraform: configuraciones inseguras

Etapa: Build

KICS, Checkov, Trivy

maestra quer un deseno, debujo o la mano ou con ferramientas.

Objetivos y requisitos

Objetivo: un pipeline DevSecOps que entregue software funcional y seguro

1. **Diseñar y después montar el pipeline CI/CD** sobre una aplicación vulnerable a propósito: primero se dibuja en dos diagramas, luego se implementa.
2. **Tres herramientas o más** de tipos distintos, con la elección justificada según la tecnología de la aplicación.
3. **Integración automática** en cada push o pull request, cada herramienta en la etapa que le corresponde.
4. **Quality gates** que detengan el pipeline ante vulnerabilidades críticas o altas, demostrando que cada herramienta detecta algo.
5. **Gestión centralizada** de los resultados en DefectDojo (u otra similar), enviados automáticamente desde el pipeline.

Objetivos y requisitos

Objetivo 1: primero diseñáis y dibujáis el pipeline; después lo montáis

a Diagrama del pipeline DevSecOps



A: modo aviso · B: bloqueante (críticas y altas)

- Etapas en orden y **herramienta** de cada etapa
- Informe que genera y cómo llega a **DefectDojo**
- Cada **quality gate**: bloqueante o aviso, y con qué **umbral**

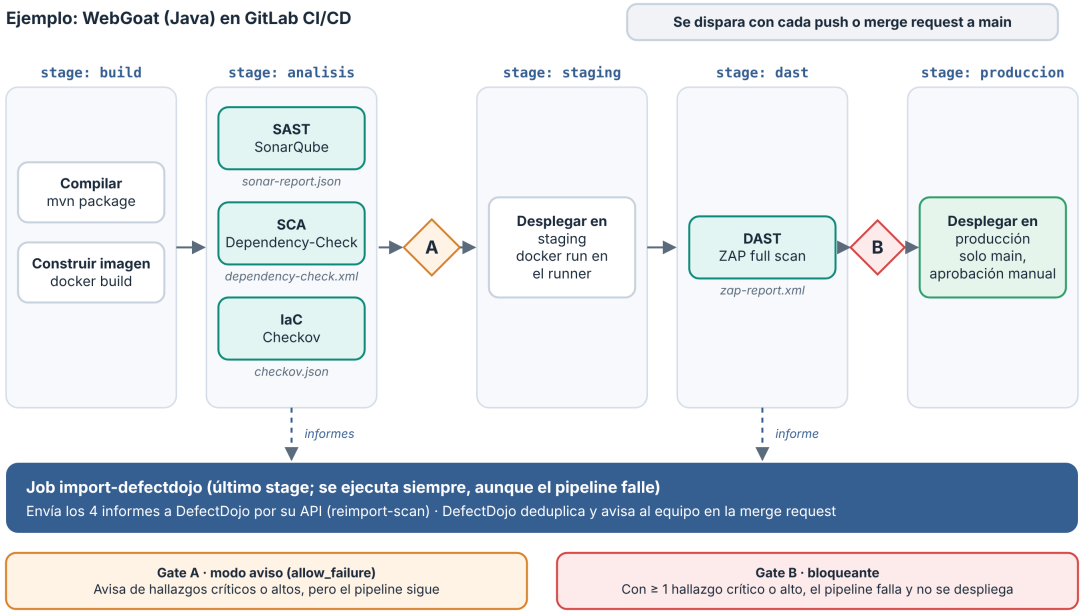
b Flujo ante una vulnerabilidad



- **Decisiones**: umbral, etapa bloqueante, ¿falso positivo?
- Qué hace el **pipeline** en cada caso y **a quién se avisa**
- Cómo se **registra y trata** el hallazgo en DefectDojo

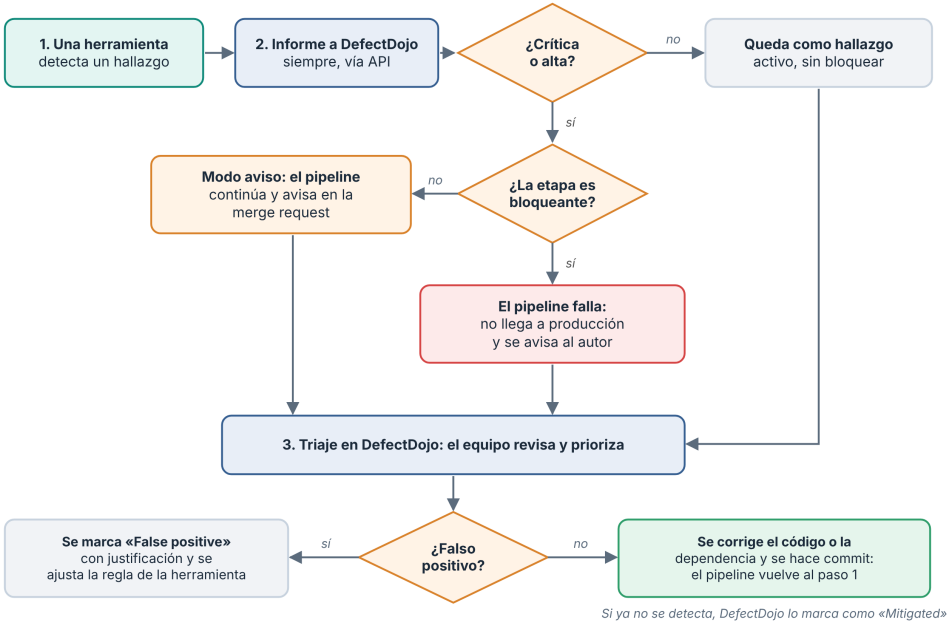
Objetivos y requisitos

Ejemplo de diagrama del pipeline (objetivo 1.a)



Objetivos y requisitos

Ejemplo de flujo ante una vulnerabilidad (objetivo 1.b)

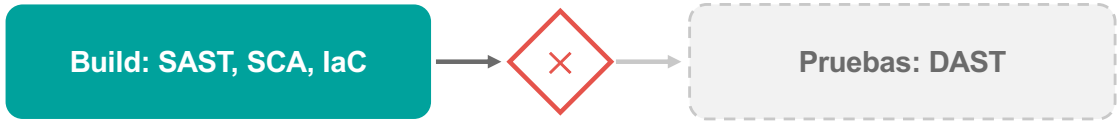


Objetivos y requisitos

Quality gates: cuándo debe detenerse el pipeline

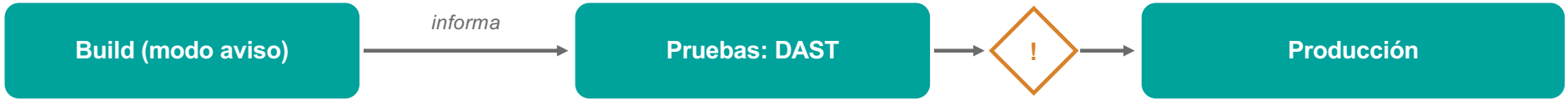
Un quality gate es una regla del tipo: «si hay vulnerabilidades críticas o altas, **el pipeline falla**».

Problema



La aplicación es vulnerable a propósito: el gate del build **siempre falla** y el DAST **nunca llega a ejecutarse**.

Solución



Modo aviso: **continue-on-error** (GitHub Actions) o **allow_failure** (GitLab CI/CD). **Al menos un gate debe bloquear de verdad.**

Objetivos y requisitos

Herramientas recomendadas (libres o con versión gratuita)

Aplicación de prueba

Juice Shop, WebGoat, DVWA, NodeGoat

CI/CD

GitHub Actions, GitLab CI/CD, Jenkins

SCA

Dependency-Check, Depex, Dependabot, Snyk

SAST

SonarQube, Semgrep, CodeQL

DAST

ZAP, Wapiti

IaC

KICS, Checkov, Trivy

IAST

Contrast, Seeker (comerciales)

Gestión

DefectDojo

Son orientativas: se puede usar cualquier otra justificándolo. Todos los enlaces están en el enunciado.

Índice

1. Contexto
2. Problemas de seguridad
3. Objetivos y requisitos
4. **Entrega**

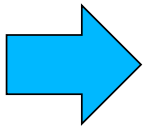
Entrega

¿Cómo y qué entrego en el PAI?

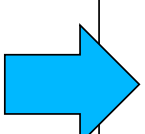
Seguimientos:
17 y 18 de Noviembre
24 y 25 de Noviembre



Especificación







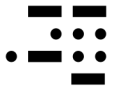
PAI4-STX.zip



Informe (PDF)



Evidencias/ Logs



Pipeline y config.



***Penalizaciones*: 10%**



Deadline: 2 de Diciembre

Entrega

¿Qué debo tener en cuenta?

- **IA**: uso ético y responsable
- **GitHub o GitLab**: el pipeline vive en el repositorio; incluid el enlace en el informe
- **Evidencias**: logs del pipeline, una ejecución que falle por un quality gate y los hallazgos en DefectDojo
- **Secretos**: nunca en el código; usad los secretos del pipeline
- **Uso responsable**: escanead solo vuestra propia aplicación
- **Documentación**: máximo 15 páginas sin portada ni anexos, con los dos diagramas del objetivo 1; cuidad el formato, el estilo y la redacción



UNIVERSIDAD
D SEVILLA
1505